

Product Requirements Document (PRD)

ICICI Bank Public Documentation Assistant using NLP and Retrieval-Augmented Generation (RAG)

Version: 1.0

Date: February 4, 2026

Document Owner: Product Management & Technical Architecture Team

Status: Draft for Review

1. Executive Summary

1.1 Overview

The ICICI Bank Public Documentation Assistant is an AI-powered chatbot that helps retail banking customers navigate complex banking procedures by providing citation-grounded, step-by-step guidance derived exclusively from publicly available ICICI Bank documentation.

1.2 Business Problem

Indian retail banking customers face significant friction when attempting routine banking tasks due to fragmented, jargon-heavy documentation scattered across multiple PDFs and web pages. This leads to:

- Failed digital workflows (Video-KYC, re-KYC updates)
- Unclear grievance escalation procedures
- Mismatched documentation between digital and branch channels
- Increased support call volume and customer frustration

1.3 Proposed Solution

A Retrieval-Augmented Generation (RAG) chatbot that:

- Ingests and indexes public ICICI Bank documentation
- Provides semantic search over vectorized document chunks
- Generates procedural answers with mandatory document citations
- Offers a streaming, conversational interface with source preview

1.4 Expected Outcomes

- **Customer Impact:** Instant access to official banking procedures without manual PDF navigation

- **Operational Impact:** Reduction in support queries for routine procedural questions
- **Technical Impact:** Scalable, maintainable RAG architecture deployable as a proof-of-concept

1.5 Target Audience

- Retail banking customers in India
- Non-technical mobile and web banking users
- Customers experiencing failed or delayed digital workflows

1.6 Project Timeline

8-week implementation cycle suitable for hackathon, academic project submission, and MS-level evaluation.

2. Problem Statement

2.1 Current State

ICICI Bank customers attempting to complete routine banking tasks encounter:

Documentation Fragmentation:

- Re-KYC, Video-KYC, grievance, and loan documentation spread across 15+ PDF documents
- Inconsistent terminology and overlapping procedures
- No centralized search or discovery mechanism

Common Failure Scenarios:

1. **Video-KYC Retry Errors:** Users receive generic error codes without clear mapping to resolution steps in official documentation
2. **Offline Re-KYC Procedures:** Customers don't know which documents are required, where to submit, or what the timeline is
3. **Grievance Escalation:** Missing information on escalation hierarchies, required reference numbers, and statutory timelines
4. **Loan/Credit-Card Documentation Mismatch:** Digital requirements differ from branch-level checklists, causing application delays
5. **Chargeback Disputes:** Users cannot find the correct dispute form, timeline limits, or supporting document requirements

2.2 User Pain Points

- **Navigation Overhead:** Users must manually download, open, and search multiple PDFs
- **Jargon Barrier:** Banking terminology (IRAC norms, NACH mandates, TAT timelines) is not explained
- **No Direct Mapping:** App error codes are not linked to official resolution procedures
- **Support Dependency:** Customers resort to calling support for basic procedural questions

2.3 Business Impact

- Increased call center volume for tier-1 procedural queries
- Customer drop-off during digital onboarding and service workflows
- Negative NPS scores related to documentation clarity
- Operational inefficiency in handling repetitive queries

2.4 Success Criteria

The solution is successful if:

- Users can obtain step-by-step procedural answers in under 10 seconds
 - Every answer includes verifiable document citations
 - Citation accuracy exceeds 95%
 - System handles 100+ concurrent users with sub-3s response latency
-

3. User Personas and Scenarios

3.1 Persona 1: Priya - Mobile-First Millennial Customer

Demographics:

- Age: 28
- Location: Mumbai
- Occupation: Marketing Manager
- Banking Behavior: Primarily mobile banking, rarely visits branches

Goals:

- Complete Video-KYC after multiple failed attempts
- Understand why her KYC verification keeps failing
- Find out if she can complete re-KYC offline

Pain Points:

- Receives "Verification Failed - Code E403" with no explanation
- Cannot find official documentation explaining error codes
- Unsure whether to retry online or visit a branch

Scenario: Priya's Video-KYC has failed three times. She asks the chatbot: "Why is my video KYC failing with error E403?" The system retrieves the relevant troubleshooting section from the Video-KYC FAQ, provides step-by-step retry instructions, and cites the official PDF (page 7).

3.2 Persona 2: Rajesh - Small Business Owner**Demographics:**

- Age: 45
- Location: Pune
- Occupation: Small business owner
- Banking Behavior: Uses both digital and branch services

Goals:

- File a chargeback dispute for an unauthorized transaction
- Understand the timeline and required documentation
- Escalate if not resolved within the promised timeframe

Pain Points:

- Cannot find the chargeback dispute form online
- Unclear on the statutory 90-day timeline for dispute filing
- Does not know how to escalate to the Banking Ombudsman

Scenario: Rajesh asks: "How do I file a chargeback for an unauthorized debit card transaction?" The chatbot provides a step-by-step procedure (login → dispute section → form submission), lists required documents (transaction statement, police FIR if applicable), cites the Chargeback Policy PDF (Section 4.2), and includes escalation timelines.

3.3 Persona 3: Anjali - First-Time Home Loan Applicant

Demographics:

- Age: 32
- Location: Bangalore
- Occupation: Software Engineer
- Banking Behavior: Prefers digital research before branch visits

Goals:

- Understand the complete list of documents required for home loan processing
- Identify differences between salaried and self-employed documentation
- Prepare documents before visiting the branch

Pain Points:

- Finds conflicting checklists on the website vs. branch printouts
- Unsure if ITR for 2 years or 3 years is required
- Does not know if property documents are needed at application stage or later

Scenario: Anjali queries: "What documents do I need to apply for a home loan as a salaried person?" The chatbot retrieves and lists the complete document checklist from the Home Loan Product Guide (Annexure A), clarifies the difference between application-stage and disbursement-stage requirements, and provides citations.

3.4 Persona 4: Vikram - Grievance Escalation Seeker

Demographics:

- Age: 52
- Location: Delhi
- Occupation: Government employee
- Banking Behavior: Prefers written communication and formal processes

Goals:

- File a grievance for delayed credit card dispute resolution
- Understand the escalation hierarchy (Branch → Nodal Officer → Principal Nodal Officer → Ombudsman)
- Obtain correct reference numbers and timelines

Pain Points:

- Grievance was filed 45 days ago with no resolution
- Does not know the next escalation level
- Cannot find the Principal Nodal Officer's contact details

Scenario: Vikram asks: "My credit card dispute has not been resolved in 30 days. How do I escalate?" The chatbot explains the escalation timeline (30 days → Nodal Officer, 30 more days → Principal Nodal Officer), provides contact details from the Grievance Redressal Policy (Section 6), and cites the RBI Ombudsman scheme timeline.

4. Scope and Non-Scope

4.1 In-Scope

4.1.1 Functional Capabilities

- **Intent-Based Query Routing:** Classify user queries into re-KYC, grievance, chargeback, loan, or credit-card intents
- **Semantic Retrieval:** Retrieve relevant document chunks using vector similarity search
- **Step-by-Step Answer Generation:** Generate procedural answers with numbered steps
- **Citation Grounding:** Attach document name, section, and page number for every answer
- **Streaming Responses:** Display answers progressively in the chat interface
- **Source Preview:** Show inline PDF/document snippets for cited sources
- **Document Refresh:** Scheduled scraping and re-indexing of public documentation
- **Fallback Handling:** Graceful responses when information is not found in the corpus

4.1.2 Document Coverage

- Re-KYC and Video-KYC guidelines
- Grievance redressal and escalation policies
- Chargeback and dispute procedures
- Home loan, personal loan, and auto loan documentation checklists
- Credit card product guides and dispute procedures
- Downloadable forms and annexures
- Public FAQs and help pages

4.1.3 Technical Infrastructure

- Next.js application (frontend + backend)
 - Cloud object storage for raw PDFs
 - Vector database (Qdrant) for embeddings
 - OpenAI API for embeddings and generation
 - Scheduled CRON job for document scraping
 - Hash-based change detection for incremental updates
 - Logging and observability for queries, retrieval, and latency
-

4.2 Out-of-Scope

4.2.1 Excluded Capabilities

- **Personalized Account Information:** No access to user account balances, transactions, or personal data
- **Transactional Operations:** Cannot execute transactions, file grievances on behalf of users, or submit forms
- **Login-Based Features:** No authentication or session management
- **Internal/Employee Documentation:** Only public, customer-facing documents are indexed
- **Multi-Language Support:** Initial release supports English only
- **Voice/Audio Interface:** Text-based chat only

4.2.2 Excluded Document Types

- Internal policy manuals
- Employee training materials
- Confidential credit assessment guidelines
- Proprietary risk models or scoring mechanisms

4.2.3 Deferred Features (Future Enhancements)

- Multi-turn contextual conversations (follow-up questions)
- User feedback and rating system
- Admin dashboard for document management
- Analytics and reporting interface
- Integration with ICICI Bank's live systems (API-based form submission)

5. Functional Requirements

5.1 Document Ingestion Pipeline

FR-1.1: Public Document Scraping

- **Description:** The system shall automatically discover and download public ICICI Bank documentation from specified URLs.
- **Acceptance Criteria:**
 - Scraper runs on a configurable CRON schedule (default: weekly)
 - Downloads PDFs, DOCs, and HTML pages from whitelisted domains
 - Stores raw files in cloud object storage (S3 or GCS) with timestamp metadata
 - Logs all download activities (success, failure, 404 errors)

FR-1.2: Change Detection

- **Description:** The system shall detect whether a document has been updated since the last ingestion.
- **Acceptance Criteria:**
 - Computes SHA-256 hash of each downloaded document
 - Compares hash with stored hash from previous ingestion
 - Flags documents as "new," "updated," or "unchanged"
 - Triggers re-indexing only for new or updated documents

FR-1.3: Document Parsing

- **Description:** The system shall extract text content from PDF and HTML documents.
- **Acceptance Criteria:**
 - Parses PDF documents using pdf-parse or similar library
 - Extracts HTML content using cheerio or similar library
 - Preserves section headings, page numbers, and table structures
 - Handles corrupted or password-protected PDFs gracefully with error logging

FR-1.4: Text Chunking

- **Description:** The system shall split documents into semantically meaningful chunks.
- **Acceptance Criteria:**

- Uses recursive character text splitter with configurable chunk size (500-1000 characters)
- Implements overlap between chunks (100-200 characters) to preserve context
- Preserves section headers as part of chunk metadata
- Generates unique chunk IDs (document_id + chunk_index)

FR-1.5: Embedding Generation

- **Description:** The system shall generate vector embeddings for each document chunk.
- **Acceptance Criteria:**
 - Uses OpenAI text-embedding-3-small or equivalent model
 - Batch processes chunks to minimize API calls
 - Stores embeddings in Qdrant with metadata (document_name, page_number, section, intent_category)
 - Implements retry logic for API failures

FR-1.6: Vector Database Upsert

- **Description:** The system shall store embeddings in Qdrant with metadata filtering support.
 - **Acceptance Criteria:**
 - Creates a single Qdrant collection for all documents
 - Upserts embeddings with metadata filters (intent_category, document_type)
 - Supports partial re-indexing (only changed documents)
 - Implements versioning (document version tracked in metadata)
-

5.2 Query Handling and Retrieval

FR-2.1: Intent Classification

- **Description:** The system shall classify user queries into predefined intent categories.
- **Acceptance Criteria:**
 - Classifies queries into: re-KYC, grievance, chargeback, loan, credit-card, or general
 - Uses lightweight LLM prompt or keyword matching for classification
 - Returns intent with confidence score
 - Defaults to "general" if no clear intent is detected

FR-2.2: Query Embedding

- **Description:** The system shall convert user queries into vector embeddings.
- **Acceptance Criteria:**
 - Uses the same embedding model as document chunks (text-embedding-3-small)
 - Generates embedding in real-time upon query submission
 - Implements caching for repeated queries (optional)

FR-2.3: Semantic Retrieval

- **Description:** The system shall retrieve the top-k most relevant document chunks.
- **Acceptance Criteria:**
 - Queries Qdrant with query embedding
 - Applies metadata filtering based on intent category
 - Returns top-5 chunks with similarity scores
 - Implements a minimum similarity threshold (e.g., 0.7) for relevance filtering

FR-2.4: Context Aggregation

- **Description:** The system shall combine retrieved chunks into a coherent context for the LLM.
 - **Acceptance Criteria:**
 - Concatenates top-k chunks with separators
 - Includes metadata (document name, page/section) for each chunk
 - Limits total context size to fit within LLM token limits (e.g., 4000 tokens)
 - Deduplicates chunks from the same document section if necessary
-

5.3 Answer Generation

FR-3.1: RAG Prompt Construction

- **Description:** The system shall construct a prompt that instructs the LLM to generate citation-grounded answers.
- **Acceptance Criteria:**
 - Prompt includes: user query, retrieved context, and citation instructions
 - Explicitly instructs LLM to provide step-by-step answers
 - Requires LLM to cite document name and page/section for each step
 - Includes fallback instruction if context does not answer the query

FR-3.2: LLM Generation

- **Description:** The system shall generate answers using OpenAI GPT-4 or equivalent model.
- **Acceptance Criteria:**
 - Uses streaming API for progressive response generation
 - Limits response length to 500-1000 tokens
 - Implements temperature control (0.3-0.5 for factual accuracy)
 - Handles API errors with retry and exponential backoff

FR-3.3: Citation Extraction

- **Description:** The system shall parse and validate citations in the generated answer.
- **Acceptance Criteria:**
 - Extracts citations in format: [Document Name, Page X, Section Y]
 - Validates that cited documents and pages exist in retrieved context
 - Flags hallucinated citations (documents not in the corpus)
 - Returns structured citation objects with answer text

FR-3.4: Fallback Response

- **Description:** The system shall provide a helpful message when no relevant information is found.
 - **Acceptance Criteria:**
 - Detects when retrieval returns no chunks above similarity threshold
 - Returns: "I couldn't find relevant information in ICICI's public documentation. Please contact ICICI customer support or visit your nearest branch."
 - Logs fallback queries for future document coverage improvements
 - Does not hallucinate or provide information outside the corpus
-

5.4 Frontend Chat Interface

FR-4.1: Chat Input

- **Description:** The system shall provide a text input field for user queries.
- **Acceptance Criteria:**
 - Text input field with placeholder: "Ask about re-KYC, grievances, loans, or chargebacks..."
 - Submit button and Enter key support

- Character limit (500 characters) to prevent abuse
- Disables input while response is streaming

FR-4.2: Message Display

- **Description:** The system shall display user queries and system responses in a chat format.
- **Acceptance Criteria:**
 - User messages aligned to the right, system messages to the left
 - Messages include timestamps
 - Supports markdown rendering for bold, lists, and links
 - Displays streaming responses with typewriter effect

FR-4.3: Citation Display

- **Description:** The system shall display citations inline with expandable source previews.
- **Acceptance Criteria:**
 - Citations displayed as superscript numbers or inline tags
 - Clicking a citation opens a source preview panel
 - Source preview shows document name, page/section, and snippet text
 - Multiple citations grouped and deduplicated

FR-4.4: Source Preview Panel

- **Description:** The system shall show a side panel with all sources cited in the current response.
- **Acceptance Criteria:**
 - Panel lists all unique sources with document name and page/section
 - Clicking a source shows the relevant text snippet
 - Provides a "Download PDF" link if the source is a PDF (links to cloud storage)
 - Panel is collapsible and responsive on mobile

FR-4.5: Error Handling

- **Description:** The system shall display user-friendly error messages for failures.
- **Acceptance Criteria:**
 - Network errors: "Unable to connect. Please check your internet connection."
 - API errors: "The system is temporarily unavailable. Please try again later."
 - Empty results: "I couldn't find information on this topic in ICICI's public documentation."

- Logs all errors for debugging
-

5.5 Logging and Observability

FR-5.1: Query Logging

- **Description:** The system shall log all user queries with metadata.
- **Acceptance Criteria:**
 - Logs query text, timestamp, intent classification, and session ID
 - Does not log any personally identifiable information (PII)
 - Stores logs in a structured format (JSON) for analysis

FR-5.2: Retrieval Logging

- **Description:** The system shall log retrieved chunks and relevance scores.
- **Acceptance Criteria:**
 - Logs document names, chunk IDs, similarity scores
 - Logs total retrieval time
 - Enables offline evaluation of retrieval quality

FR-5.3: Generation Logging

- **Description:** The system shall log LLM generation metadata.
- **Acceptance Criteria:**
 - Logs token usage (prompt tokens, completion tokens)
 - Logs generation latency
 - Logs citation extraction success/failure
 - Enables cost tracking and performance monitoring

FR-5.4: Ingestion Audit Logs

- **Description:** The system shall log all document ingestion activities.
- **Acceptance Criteria:**
 - Logs scraping success/failure for each URL
 - Logs hash changes (new, updated, unchanged documents)
 - Logs parsing and chunking errors
 - Logs embedding and upsert counts

6. Non-Functional Requirements

6.1 Performance

NFR-1.1: Response Latency

- **Requirement:** End-to-end query response time (from user submission to first token streamed) shall be under 3 seconds for 95% of queries.
- **Measurement:** P95 latency tracked via server-side logs.
- **Dependencies:** Qdrant query performance, OpenAI API latency.

NFR-1.2: Streaming Responsiveness

- **Requirement:** Streaming responses shall begin rendering within 1 second of query submission.
- **Measurement:** Time to first byte (TTFB) for streaming endpoint.

NFR-1.3: Concurrent Users

- **Requirement:** System shall handle 100 concurrent users without degradation.
- **Measurement:** Load testing with simulated concurrent sessions.

NFR-1.4: Document Ingestion Time

- **Requirement:** Full re-indexing of 100 documents shall complete within 30 minutes.
- **Measurement:** End-to-end ingestion pipeline duration.

6.2 Scalability

NFR-2.1: Document Corpus Growth

- **Requirement:** System shall support up to 500 documents (approximately 5,000 chunks) without architectural changes.
- **Mitigation:** Qdrant is horizontally scalable if corpus grows beyond this limit.

NFR-2.2: Query Volume

- **Requirement:** System shall handle up to 10,000 queries per day.
- **Mitigation:** Rate limiting and caching for repeated queries (future enhancement).

6.3 Reliability

NFR-3.1: Uptime

- **Requirement:** System shall maintain 99% uptime during business hours (9 AM - 9 PM IST).
- **Measurement:** Health check endpoint monitored every 5 minutes.

NFR-3.2: Error Handling

- **Requirement:** All API failures (OpenAI, Qdrant, cloud storage) shall be handled gracefully with retry logic.
- **Implementation:** Exponential backoff with maximum 3 retries.

NFR-3.3: Data Integrity

- **Requirement:** Document ingestion failures shall not corrupt the existing index.
 - **Implementation:** Atomic upsert operations; rollback on failure.
-

6.4 Usability

NFR-4.1: Response Clarity

- **Requirement:** Generated answers shall be structured as numbered steps when procedural guidance is requested.
- **Measurement:** Manual evaluation of answer quality.

NFR-4.2: Citation Visibility

- **Requirement:** 100% of answers shall include at least one citation from the corpus.
- **Measurement:** Automated validation in generation pipeline.

NFR-4.3: Mobile Responsiveness

- **Requirement:** Chat interface shall be fully functional on mobile devices (viewport width $\geq 360\text{px}$).
 - **Measurement:** Cross-browser testing on iOS Safari and Android Chrome.
-

6.5 Security and Privacy

NFR-5.1: No PII Storage

- **Requirement:** System shall not collect, store, or process any personally identifiable information.
- **Implementation:** No login, no session tracking beyond anonymized query logs.

NFR-5.2: Public Data Only

- **Requirement:** System shall ingest only publicly accessible documents (no authentication required for download).
- **Implementation:** Whitelist of public URLs; reject any authentication-gated content.

NFR-5.3: API Key Security

- **Requirement:** OpenAI and Qdrant API keys shall be stored as environment variables, never hardcoded.
 - **Implementation:** Use `.env.local` file (excluded from version control).
-

6.6 Maintainability

NFR-6.1: Code Quality

- **Requirement:** Code shall follow consistent JavaScript style guidelines (ESLint).
- **Measurement:** Automated linting in pre-commit hooks.

NFR-6.2: Documentation

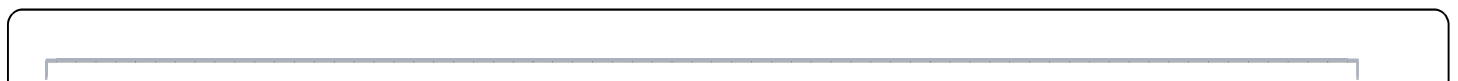
- **Requirement:** All major functions and API routes shall include JSDoc comments.
- **Measurement:** Code review checklist.

NFR-6.3: Modularity

- **Requirement:** Ingestion pipeline and query pipeline shall be independently testable.
 - **Implementation:** Separate modules for scraping, parsing, embedding, retrieval, and generation.
-

7. System Architecture Overview

7.1 High-Level Architecture Diagram



USER INTERFACE

Next.js Frontend (React + Tailwind CSS)

- Chat Input
- Message Display (Streaming)
- Citation Display
- Source Preview Panel



HTTP/SSE



NEXT.JS API ROUTES

/api/chat (Query Handling)

- Intent Classification
- Query Embedding
- Semantic Retrieval
- RAG Generation (Streaming)
- Citation Extraction

/api/ingest (Document Ingestion - CRON)

- Scrape Public URLs
- Download PDFs/HTML
- Hash-Based Change Detection
- Parse → Chunk → Embed → Upsert



Qdrant

OpenAI API

Cloud

Vector DB

- Embeddings

Storage

- GPT-4

(S3/GCS)

- Chunks

- Streaming

- Metadata

- Raw PDFs

- Filters

- Versioning

7.2 Component Breakdown

7.2.1 Frontend (Next.js React Components)

- **ChatInterface.jsx:** Main chat UI with message list and input field
- **MessageBubble.jsx:** Individual message component (user/system)
- **CitationTag.jsx:** Inline citation component with click handler
- **SourcePreview.jsx:** Side panel for displaying cited sources
- **StreamingText.jsx:** Component for rendering streaming LLM responses

7.2.2 Backend (Next.js API Routes)

- **/api/chat:** Main query endpoint
 - Input: `{ query: string }`
 - Output: Server-Sent Events (SSE) stream with tokens and citations
- **/api/ingest:** Document ingestion endpoint (triggered by CRON)
 - Input: None (runs on schedule)
 - Output: Ingestion status and audit logs

7.2.3 Core Modules (lib/)

- **scraper.js:** Functions for downloading public documents
- **parser.js:** PDF and HTML parsing utilities
- **chunker.js:** Text splitting and chunking logic
- **embedder.js:** OpenAI embedding generation
- **qdrant-client.js:** Qdrant vector database operations
- **intent-classifier.js:** Query intent detection
- **rag-pipeline.js:** Retrieval and generation orchestration
- **citation-extractor.js:** Parse and validate citations from LLM output

7.2.4 External Services

- **Qdrant Cloud:** Managed vector database
- **OpenAI API:** Embeddings (text-embedding-3-small) and generation (GPT-4)
- **S3/GCS:** Object storage for raw PDFs
- **Vercel Cron:** Scheduled document scraping (or equivalent)

7.3 Data Flow

7.3.1 Offline Ingestion Flow

1. **Scraper** downloads public documents from whitelisted URLs → stores in cloud storage
2. **Change Detector** computes hashes → identifies new/updated documents
3. **Parser** extracts text from PDFs/HTML
4. **Chunker** splits text into 500-1000 character chunks with overlap
5. **Embedder** generates vector embeddings for each chunk
6. **Qdrant Client** upserts embeddings with metadata (document name, page, section, intent)

7.3.2 Online Query Flow

1. **User** submits query via chat interface
2. **Intent Classifier** categorizes query (re-KYC, grievance, loan, etc.)
3. **Embedder** generates query embedding
4. **Qdrant Client** retrieves top-5 similar chunks (filtered by intent)
5. **RAG Pipeline** constructs prompt with retrieved context
6. **OpenAI API** streams generated response
7. **Citation Extractor** parses citations from response
8. **Frontend** displays answer with inline citations and source preview

7.4 Technology Stack Summary

Layer	Technology
Frontend	Next.js (React), Tailwind CSS
Backend	Next.js API Routes (Node.js)
Vector Database	Qdrant Cloud
Embeddings	OpenAI text-embedding-3-small
Generation	OpenAI GPT-4 (streaming)
Object Storage	AWS S3 or Google Cloud Storage

Layer	Technology
Scheduling	Vercel Cron or Node-Cron
Language	JavaScript (ES6+)
Styling	Tailwind CSS
Deployment	Vercel (or equivalent serverless)

8. Data Acquisition and Ingestion Workflow

8.1 Document Discovery and Scraping

8.1.1 Source Configuration

- Maintain a whitelist of public ICICI Bank URLs in a configuration file (`config/document-sources.json`)
- Categories:
 - Re-KYC and Video-KYC: `https://www.icicibank.com/customer-service/re-kyc`
 - Grievance Redressal: `https://www.icicibank.com/customer-service/grievance-redressal`
 - Chargebacks: `https://www.icicibank.com/customer-service/chargeback-policy`
 - Loans: `https://www.icicibank.com/personal-banking/loans`
 - Credit Cards: `https://www.icicibank.com/personal-banking/cards`

8.1.2 Scraping Logic

- Use `axios` or `node-fetch` for HTTP requests
- Use `cheerio` for HTML parsing (extract PDF links)
- Use `pdf-parse` for PDF text extraction
- Implement retry logic (3 attempts with exponential backoff)
- Handle SSL/TLS errors and timeouts gracefully

8.1.3 Storage

- Download PDFs to cloud storage (S3 bucket: `icici-docs-raw`)
- Naming convention: `{category}/{document-name}_{timestamp}.pdf`
- Maintain metadata file: `metadata.json` with document URLs, download timestamps, and hashes

8.2 Change Detection

8.2.1 Hash Computation

- Compute SHA-256 hash of each downloaded document
- Store hash in metadata file alongside document path

8.2.2 Comparison Logic

- On each scraping run, compare current hash with previous hash
- If hash matches: Skip document (no re-indexing needed)
- If hash differs or document is new: Flag for re-indexing
- Delete removed documents from Qdrant

8.2.3 Partial Re-Indexing

- Only process flagged documents
 - Avoids redundant embedding API calls and reduces cost
-

8.3 Document Parsing

8.3.1 PDF Parsing

- Use `pdf-parse` library
- Extract text content, page numbers, and section headings
- Handle multi-column layouts by parsing in reading order
- Fallback: OCR for scanned PDFs (using Tesseract.js if needed)

8.3.2 HTML Parsing

- Use `cheerio` to extract main content (strip navigation, footers)
- Preserve heading hierarchy (H1, H2, H3)
- Extract FAQ lists and table structures

8.3.3 Text Preprocessing

- Remove excessive whitespace and line breaks
- Normalize Unicode characters
- Remove page headers/footers (e.g., "ICICI Bank Limited | Page 3")

8.4 Text Chunking

8.4.1 Chunking Strategy

- **Method:** Recursive character text splitter
- **Chunk Size:** 800 characters
- **Overlap:** 150 characters
- **Separator Hierarchy:** `\n\n` (paragraph) → `\n` (line) → `.` (sentence) → (word)

8.4.2 Metadata Preservation

- Each chunk includes:
 - `document_id`: Unique identifier for source document
 - `document_name`: Human-readable document name
 - `page_number`: Page number in source PDF
 - `section`: Section heading from document hierarchy
 - `intent_category`: Manual tag (re-KYC, grievance, loan, card, chargeback)
 - `chunk_index`: Position of chunk within document

8.4.3 Section-Aware Chunking

- If a section heading is detected, prepend it to the first chunk of that section
 - Example: "Section 4.2: Chargeback Dispute Process" prepended to chunks from that section
-

8.5 Embedding Generation

8.5.1 Model Selection

- **Model:** OpenAI `text-embedding-3-small` (1536 dimensions)
- **Rationale:** Cost-effective, high-quality embeddings for English text

8.5.2 Batch Processing

- Batch chunks in groups of 100 to minimize API calls
- Implement rate limiting (100 requests per minute to avoid OpenAI limits)

8.5.3 Error Handling

- Retry failed embedding requests up to 3 times
 - Log failed chunks for manual review
 - Do not halt the entire pipeline on single chunk failure
-

8.6 Vector Database Upsert

8.6.1 Qdrant Collection Setup

- **Collection Name:** `icici_bank_docs`
- **Vector Dimension:** 1536
- **Distance Metric:** Cosine similarity
- **Metadata Schema:**

json

```
{
  "document_id": "string",
  "document_name": "string",
  "page_number": "integer",
  "section": "string",
  "intent_category": "string",
  "chunk_index": "integer",
  "source_url": "string",
  "ingestion_timestamp": "datetime"
}
```

8.6.2 Upsert Logic

- Use `upsert` operation (overwrite if chunk_id already exists)
- Chunk ID format: `{document_id}_{chunk_index}`
- Enables partial re-indexing without duplication

8.6.3 Indexing Optimization

- Enable HNSW index for fast approximate nearest neighbor search
 - Set `ef_construct: 128` and `m: 16` for balanced speed/accuracy
-

8.7 Ingestion Pipeline Orchestration

8.7.1 CRON Schedule

- Run ingestion pipeline weekly (every Sunday at 2 AM IST)
- Use Vercel Cron or `node-cron` for scheduling

8.7.2 Pipeline Steps (Sequential)

1. Scrape documents from configured URLs
2. Download new/updated documents to cloud storage
3. Compute hashes and detect changes
4. Parse changed documents
5. Chunk text content
6. Generate embeddings in batches
7. Upsert to Qdrant
8. Log ingestion report (documents processed, chunks created, errors)

8.7.3 Error Recovery

- If pipeline fails mid-execution, resume from last successful step
 - Maintain checkpoint state in metadata file
-

8.8 Audit Logging

8.8.1 Log Structure

```
json
```

```
{
  "timestamp": "2026-02-04T02:00:00Z",
  "run_id": "uuid",
  "documents_scraped": 12,
  "documents_new": 2,
  "documents_updated": 1,
  "documents_unchanged": 9,
  "chunks_created": 245,
  "chunks_upserted": 245,
  "errors": [
    {
      "document_id": "loan-guide-2026",
      "error": "PDF parsing failed",
      "details": "Corrupted file"
    }
  ],
  "duration_seconds": 1824
}
```

8.8.2 Storage

- Store logs in cloud storage (`s3://icici-docs-logs/ingestion/`)
 - Retain logs for 90 days
-

9. Retrieval and Generation Workflow

9.1 Query Preprocessing

9.1.1 Input Validation

- Trim leading/trailing whitespace
- Reject queries shorter than 5 characters (return error message)
- Reject queries longer than 500 characters (truncate with warning)

9.1.2 Query Normalization

- Convert to lowercase for intent classification
 - Remove special characters (preserve question marks)
-

9.2 Intent Classification

9.2.1 Classification Method

- **Option 1 (Lightweight):** Keyword-based classification
 - Keywords for "re-KYC": kyc, video kyc, re-kyc, verification, documents
 - Keywords for "grievance": complaint, escalation, nodal officer, ombudsman
 - Keywords for "chargeback": dispute, chargeback, unauthorized transaction
 - Keywords for "loan": loan, home loan, personal loan, auto loan, documents
 - Keywords for "credit-card": credit card, card dispute, card application
- **Option 2 (LLM-Based):** Few-shot prompt to GPT-4
 - Prompt: "Classify the following banking query into one of: re-KYC, grievance, chargeback, loan, credit-card, general. Query: {query}"
 - Response: Intent label

9.2.2 Fallback

- If no intent is detected with confidence > 0.6 , classify as "general"
 - "General" intent retrieves from all document categories
-

9.3 Query Embedding

9.3.1 Embedding Generation

- Use OpenAI `text-embedding-3-small` (same model as document chunks)
 - Generate embedding for the normalized query
 - Cache embeddings for repeated queries (optional optimization)
-

9.4 Semantic Retrieval

9.4.1 Vector Search

- Query Qdrant with:
 - Query embedding
 - Intent filter: `intent_category = {classified_intent}` (if not "general")
 - Limit: Top-5 chunks

- Minimum similarity threshold: 0.70

9.4.2 Ranking

- Qdrant returns chunks sorted by cosine similarity (descending)
- No additional re-ranking in MVP; future enhancement: cross-encoder re-ranking

9.4.3 Fallback

- If no chunks exceed similarity threshold: Return empty context
 - Trigger fallback response generation
-

9.5 Context Aggregation

9.5.1 Context Construction

- Concatenate top-5 chunks with separators
- Format:

```
[Document: {document_name}, Page: {page_number}, Section: {section}]  
{chunk_text}
```

```
[Document: {document_name}, Page: {page_number}, Section: {section}]  
{chunk_text}
```

9.5.2 Token Limit Management

- Limit total context to 4000 tokens (to fit within GPT-4 context window)
 - If context exceeds limit, truncate from the bottom (keep top-ranked chunks)
-

9.6 RAG Prompt Construction

9.6.1 System Prompt

You are an informational assistant for ICICI Bank customers. Your role is to provide step-by-step procedural guidance based on official ICICI Bank documentation.

Rules:

1. Only use information from the provided context. Do not use external knowledge.
2. Always provide step-by-step instructions when answering procedural questions.
3. Cite the source document name, page number, and section for each piece of information.
4. If the context does not contain the answer, respond: "I couldn't find this information in ICICI's public documentation. Please contact ICICI customer support."
5. Format citations as: [Source: Document Name, Page X, Section Y]
6. Be concise and avoid jargon. Use simple language.

9.6.2 User Prompt

Context:

{aggregated_context}

User Question:

{user_query}

Please provide a step-by-step answer with citations.

9.7 LLM Generation

9.7.1 API Configuration

- **Model:** `gpt-4-turbo` (or `gpt-4`)
- **Temperature:** 0.3 (low temperature for factual accuracy)
- **Max Tokens:** 1000
- **Streaming:** Enabled (Server-Sent Events)
- **Top-p:** 0.9

9.7.2 Streaming Implementation

- Use OpenAI streaming API
- Send tokens to frontend via Server-Sent Events (SSE)
- Frontend renders tokens progressively (typewriter effect)

9.7.3 Error Handling

- OpenAI API timeout (30 seconds): Return error message
 - Rate limit errors: Retry with exponential backoff
 - Invalid API key: Log error and return generic failure message
-

9.8 Citation Extraction and Validation

9.8.1 Citation Parsing

- Use regex to extract citations in format: `[Source: (.+?), Page (.+?), Section (.+?)]`
- Parse into structured objects:

```
javascript
```

```
{  
  document_name: "Re-KYC Guidelines",  
  page_number: 7,  
  section: "Section 4.2",  
  snippet: "Customers can complete re-KYC online via Video-KYC..."  
}
```

9.8.2 Validation

- Cross-check cited document names against retrieved chunks
- Flag citations that reference documents not in the retrieved context (hallucinations)
- Log validation failures for monitoring

9.8.3 Fallback for Missing Citations

- If LLM response contains no citations, append a warning:
 - "Note: This answer may not be fully citation-grounded. Please verify with official ICICI documentation."
-

9.9 Response Delivery

9.9.1 Streaming Response

- Stream answer text tokens to frontend via SSE
- After streaming completes, send citation objects as a final event

9.9.2 Citation Rendering

- Frontend displays citations as inline superscript numbers
- Clicking a citation opens the source preview panel with document snippet

9.9.3 Fallback Response

- If no chunks retrieved or context is insufficient:
 - Return: "I couldn't find relevant information in ICICI's public documentation. For personalized assistance, please contact ICICI customer support at 1860 120 7777 or visit your nearest branch."
-

10. Frontend Experience and Interaction Design

10.1 Chat Interface Layout

10.1.1 Overall Structure

- **Header:** "ICICI Bank Documentation Assistant" with disclaimer
- **Chat Area:** Scrollable message list (80% of viewport height)
- **Input Area:** Text input field + submit button (fixed at bottom)
- **Source Panel:** Collapsible side panel for citations (desktop) or modal (mobile)

10.1.2 Responsive Design

- Desktop ($\geq 1024\text{px}$): Chat area (60%) + Source panel (40%)
 - Tablet (768px-1023px): Chat area (70%) + Collapsible source panel
 - Mobile ($< 768\text{px}$): Full-width chat area + Modal source preview
-

10.2 Message Display

10.2.1 User Messages

- Aligned to the right
- Light blue background (`background-color: #E3F2FD`)
- Rounded corners (border-radius: 12px)
- Timestamp below message (gray, small font)

10.2.2 System Messages

- Aligned to the left
- White background with light gray border
- Rounded corners (border-radius: 12px)
- Timestamp below message

10.2.3 Streaming Effect

- Display tokens progressively as they arrive from SSE
 - Blinking cursor at the end of incomplete messages
 - Smooth scrolling to keep latest message in view
-

10.3 Citation Display

10.3.1 Inline Citations

- Display as superscript numbers: "You can complete re-KYC online^[1] or visit a branch^[2]."
- Clicking a citation number highlights the source in the side panel

10.3.2 Citation Grouping

- If multiple citations reference the same document and page, group them as "[1,2]"

10.3.3 Visual Design

- Citation numbers in blue ( #1976D2)
 - Hover effect: Tooltip shows document name and section
-

10.4 Source Preview Panel

10.4.1 Content

- List of all unique sources cited in the current response
- For each source:
 - Document name
 - Page/section
 - Text snippet (200-300 characters)
 - "Download PDF" link (if available)

10.4.2 Interaction

- Clicking a source scrolls to the relevant snippet
- Clicking "Download PDF" opens the document in a new tab (from cloud storage)

10.4.3 Collapsible Behavior

- Desktop: Toggle button to show/hide panel
 - Mobile: Modal that overlays the chat area
-

10.5 Input Experience

10.5.1 Placeholder Text

- "Ask about re-KYC, grievances, loans, chargebacks, or credit cards..."

10.5.2 Character Counter

- Show character count near input field (e.g., "120/500")
- Turn red when approaching limit

10.5.3 Submit Behavior

- Disable submit button while response is streaming
- Show loading spinner on button during query processing

10.5.4 Suggested Queries (Optional Enhancement)

- Display 3-5 example queries below input field:
 - "How do I complete Video-KYC?"
 - "What documents are needed for a home loan?"
 - "How do I file a chargeback dispute?"
-

10.6 Error and Empty States

10.6.1 No Results Found

- Message: "I couldn't find information on this topic in ICICI's public documentation. Please contact ICICI customer support at 1860 120 7777."

- Display in a distinct info box (yellow background)

10.6.2 Network Error

- Message: "Unable to connect to the server. Please check your internet connection and try again."
- Display with a retry button

10.6.3 API Error

- Message: "The system is temporarily unavailable. Please try again in a few moments."
- Log error details server-side for debugging

10.6.4 Empty Chat State

- Display welcome message:
 - "Welcome to the ICICI Bank Documentation Assistant. Ask me about re-KYC, grievances, loans, chargebacks, or credit cards."
 - Show example queries
-

10.7 Disclaimer and Legal Notice

10.7.1 Header Disclaimer

- Display prominently at the top of the page:
 - "This is an informational assistant based on public ICICI Bank documentation. For account-specific queries or transactions, please contact ICICI customer support or visit your nearest branch."

10.7.2 Footer Disclaimer

- Small text at the bottom:
 - "This assistant provides general information only and is not an official ICICI Bank service. Always verify critical information with ICICI Bank directly."
-

10.8 Accessibility

10.8.1 Keyboard Navigation

- Tab navigation through input field, submit button, and citations
- Enter key submits query

- Escape key closes source preview panel

10.8.2 Screen Reader Support

- ARIA labels for chat messages, citations, and source preview
- Alt text for icons (send button, citation numbers)

10.8.3 Color Contrast

- Ensure WCAG AA compliance (contrast ratio $\geq 4.5:1$ for text)
-

11. Security, Privacy and Compliance

11.1 Data Privacy

11.1.1 No PII Collection

- The system does not collect, store, or process:
 - Account numbers
 - Customer IDs
 - Transaction details
 - Personal names, addresses, or contact information
- Query logs are anonymized (no session IDs linked to user identity)

11.1.2 Session Management

- No user login or authentication
- No persistent sessions
- No cookies except for essential functionality (e.g., CSRF protection)

11.1.3 Data Retention

- Query logs retained for 30 days (for debugging and evaluation)
 - Logs do not contain PII
 - After 30 days, logs are automatically deleted
-

11.2 Document Security

11.2.1 Public Documents Only

- Only publicly accessible documents are ingested
- No internal bank policies, employee manuals, or confidential documents
- Scraper validates that documents are accessible without authentication

11.2.2 Document Versioning

- Each document version is timestamped
- Old versions are archived (not deleted) for audit purposes

11.2.3 Access Control

- Cloud storage bucket (S3/GCS) is private (not publicly accessible)
 - Only the ingestion pipeline has write access
 - Frontend serves documents via signed URLs (read-only, time-limited)
-

11.3 API Security

11.3.1 API Key Management

- OpenAI and Qdrant API keys stored as environment variables
- Never hardcoded in source code
- Never exposed to the frontend

11.3.2 Rate Limiting

- Implement rate limiting on `/api/chat` endpoint (10 requests per minute per IP)
- Prevents abuse and DoS attacks

11.3.3 Input Validation

- Sanitize user inputs to prevent injection attacks
 - Reject queries with suspicious patterns (SQL injection, XSS)
-

11.4 Compliance

11.4.1 Disclaimer Requirement

- Clearly state that the assistant is not an official ICICI Bank service

- Users are advised to verify information with ICICI Bank directly

11.4.2 Terms of Use

- Display terms of use on first visit (optional)
- Terms include:
 - No liability for inaccuracies
 - No guarantee of real-time updates
 - No personalized financial advice

11.4.3 Copyright

- Document content is owned by ICICI Bank
 - System displays attribution for all cited documents
 - No redistribution of copyrighted material outside the application
-

11.5 Ethical Considerations

11.5.1 Accuracy and Truthfulness

- System must not hallucinate or fabricate information
- All answers must be citation-grounded
- When uncertain, the system defers to ICICI customer support

11.5.2 Bias Mitigation

- LLM responses are based solely on ICICI documentation (no external bias)
- No personalization that could introduce discriminatory patterns

11.5.3 Transparency

- Users are informed that they are interacting with an AI assistant
 - Citations make the information source transparent
-

12. Risks and Mitigation

12.1 Technical Risks

Risk	Impact	Probability	Mitigation
OpenAI API Downtime	High (system unusable)	Medium	Implement retry logic with exponential backoff; cache frequent queries; display graceful error message
Qdrant Vector DB Failure	High (retrieval fails)	Low	Use Qdrant Cloud (managed service with 99.9% SLA); implement health checks; fallback to error message
Document Scraping Failures	Medium (stale data)	Medium	Retry failed scrapes; log failures; send alerts for repeated failures; manual backup ingestion
Slow Embedding Generation	Medium (ingestion delays)	Low	Batch processing; parallel API calls; use smaller embedding model if needed
Citation Hallucinations	High (user misinformation)	Medium	Validate citations against retrieved chunks; flag unverified citations; log hallucination rate for monitoring

12.2 Data Quality Risks

Risk	Impact	Probability	Mitigation
Outdated Documentation	High (incorrect guidance)	High	Scheduled weekly scraping; hash-based change detection; display document version/timestamp in UI
Incomplete Documentation	Medium (missing answers)	Medium	Clearly communicate fallback to customer support; track "no results" queries to identify coverage gaps
Parsing Errors (Corrupted PDFs)	Medium (missing content)	Low	Log parsing failures; manual review of failed documents; fallback to HTML sources if available
Chunking Artifacts	Low (context loss)	Medium	Use overlap between chunks; test chunking strategy on sample documents; manual evaluation

12.3 User Experience Risks

Risk	Impact	Probability	Mitigation
User Misinterprets AI as Official Channel	High (legal/trust issues)	High	Prominent disclaimers; clear branding as "informational assistant"; avoid ICICI branding unless authorized
Slow Response Times	Medium (user frustration)	Medium	Optimize retrieval (use Qdrant indexing); implement caching; set expectation with loading indicators
Mobile Usability Issues	Medium (reduced adoption)	Low	Responsive design testing; touch-friendly UI; mobile-first development

12.4 Legal and Compliance Risks

Risk	Impact	Probability	Mitigation
Copyright Violation	High (legal action)	Low	Use only public documents; provide attribution; do not redistribute content outside application
Misleading Financial Advice	High (user financial harm)	Medium	Explicitly state "not financial advice"; focus on procedural guidance only; defer to ICICI for account-specific queries
Privacy Violation	High (regulatory penalties)	Low	No PII collection; anonymized logs; comply with data retention limits

12.5 Operational Risks

Risk	Impact	Probability	Mitigation
High API Costs (OpenAI)	Medium (budget overrun)	Medium	Monitor token usage; implement query length limits; cache frequent queries; set daily budget alerts
Scaling Beyond MVP Capacity	Medium (performance degradation)	Low	Load testing before production; horizontal scaling of Qdrant; CDN for static assets
Insufficient Monitoring	Medium (undetected failures)	Medium	Implement logging for all critical paths; set up alerts for error spikes; weekly log reviews

13. Success Metrics and Evaluation Plan

13.1 Primary Success Metrics

13.1.1 Citation Accuracy

- **Definition:** Percentage of cited documents that exist in the corpus and accurately support the generated answer.
- **Target:** $\geq 95\%$
- **Measurement:** Manual evaluation of 100 randomly sampled queries.
- **Evaluation Rubric:**
 - 1 point: Citation exists and supports the answer
 - 0.5 points: Citation exists but is only partially relevant
 - 0 points: Citation does not exist or is irrelevant

13.1.2 Retrieval Relevance

- **Definition:** Percentage of retrieved chunks that are relevant to the user query.
- **Target:** $\geq 80\%$ of top-3 retrieved chunks are relevant
- **Measurement:** Manual evaluation of 100 queries.
- **Evaluation Rubric:**
 - Relevant: Chunk contains information that helps answer the query
 - Irrelevant: Chunk is off-topic or does not contribute to the answer

13.1.3 Answer Completeness

- **Definition:** Percentage of answers that provide complete step-by-step guidance for procedural queries.
 - **Target:** $\geq 85\%$
 - **Measurement:** Manual evaluation of 100 procedural queries.
 - **Evaluation Rubric:**
 - Complete: All necessary steps are included and in the correct order
 - Partially complete: Some steps are missing or unclear
 - Incomplete: Critical steps are missing
-

13.2 Performance Metrics

13.2.1 End-to-End Latency

- **Definition:** Time from query submission to first token streamed.
- **Target:** P95 latency <3 seconds
- **Measurement:** Server-side logging of query processing time.

13.2.2 Retrieval Latency

- **Definition:** Time to retrieve top-k chunks from Qdrant.
- **Target:** P95 <500ms
- **Measurement:** Qdrant query duration logs.

13.2.3 Generation Latency

- **Definition:** Time to generate the full response (excluding streaming).
 - **Target:** P95 <2 seconds
 - **Measurement:** OpenAI API call duration.
-

13.3 User Experience Metrics

13.3.1 Fallback Rate

- **Definition:** Percentage of queries that trigger the "no information found" fallback.
- **Target:** <10%
- **Measurement:** Count of fallback responses / total queries.

13.3.2 User Satisfaction (Simulated)

- **Definition:** Average rating from simulated user evaluations.
- **Target:** $\geq 4.0/5.0$
- **Measurement:** Test users rate 50 interactions on a 5-point scale (1=poor, 5=excellent).

13.3.3 Citation Coverage

- **Definition:** Percentage of answers that include at least one citation.
- **Target:** 100%
- **Measurement:** Automated validation in generation pipeline.

13.4 Operational Metrics

13.4.1 Token Usage (OpenAI)

- **Definition:** Average tokens consumed per query (prompt + completion).
- **Target:** <5,000 tokens per query
- **Measurement:** OpenAI API usage logs.

13.4.2 Ingestion Success Rate

- **Definition:** Percentage of documents successfully ingested in each CRON run.
- **Target:** $\geq 95\%$
- **Measurement:** Ingestion pipeline logs (documents processed / documents attempted).

13.4.3 Document Freshness

- **Definition:** Age of the oldest document in the corpus.
 - **Target:** <7 days
 - **Measurement:** Timestamp comparison in metadata file.
-

13.5 Evaluation Plan

13.5.1 Phase 1: Unit Testing (Week 4)

- **Objective:** Validate individual components (parser, chunker, retriever, generator).
- **Method:**
 - Test PDF parsing on 10 sample documents
 - Verify chunking output (chunk size, overlap, metadata)
 - Test retrieval with synthetic queries
 - Test LLM generation with mock retrieved context

13.5.2 Phase 2: Integration Testing (Week 6)

- **Objective:** Validate end-to-end pipeline.
- **Method:**
 - Run 50 test queries through the full pipeline
 - Verify that answers are citation-grounded

- Check streaming behavior and frontend rendering

13.5.3 Phase 3: User Evaluation (Week 8)

- **Objective:** Assess user experience and answer quality.
- **Method:**
 - Recruit 10 test users (mix of banking customers and non-experts)
 - Provide 5 sample queries per user (total: 50 queries)
 - Collect ratings on:
 - Answer clarity (1-5 scale)
 - Citation usefulness (1-5 scale)
 - Overall satisfaction (1-5 scale)
 - Collect qualitative feedback

13.5.4 Phase 4: Benchmarking (Week 8)

- **Objective:** Compare against baseline (manual PDF search).
 - **Method:**
 - Measure time for users to find answers manually vs. using the chatbot
 - Baseline: Average 5-10 minutes to locate answer in PDFs
 - Target: <1 minute using chatbot
-

13.6 Continuous Monitoring

13.6.1 Dashboard Metrics (Weekly Review)

- Total queries processed
- Average latency (P50, P95)
- Fallback rate
- Citation accuracy (sampled)
- Error rate (API failures, parsing errors)

13.6.2 Alerts

- Error rate >5% (immediate alert)
- P95 latency >5 seconds (daily alert)

- Ingestion failures for 2+ consecutive runs (immediate alert)
-

14. Implementation Phases and Timeline (8 Weeks)

Week 1: Project Setup and Infrastructure

Objectives:

- Set up development environment
- Configure cloud services
- Implement basic Next.js structure

Tasks:

1. Initialize Next.js project with App Router (JavaScript)
2. Set up Tailwind CSS
3. Create Qdrant Cloud account and collection
4. Set up OpenAI API account
5. Set up cloud storage (S3 or GCS) for PDFs
6. Configure environment variables (`.env.local`)
7. Create basic project structure:

```
/app
/api
  /chat
  /ingest
/components
/lib
/config
/public
```

Deliverables:

- Functional Next.js dev server
 - Configured cloud services (Qdrant, OpenAI, S3/GCS)
 - README with setup instructions
-

Week 2: Document Scraping and Ingestion Pipeline

Objectives:

- Implement document scraping
- Build parsing and chunking logic

Tasks:

1. Create `lib/scrapper.js`:
 - Download PDFs from whitelisted URLs
 - Upload to cloud storage
 - Compute SHA-256 hashes
2. Create `lib/parser.js`:
 - PDF text extraction (pdf-parse)
 - HTML content extraction (cheerio)
3. Create `lib/chunker.js`:
 - Recursive text splitter (800 chars, 150 overlap)
 - Metadata attachment (document_id, page, section, intent)
4. Create `config/document-sources.json`:
 - List of public ICICI Bank URLs (mock for now)
5. Implement `/api/ingest` route:
 - Orchestrate scraping → parsing → chunking
 - Log ingestion results

Deliverables:

- Functional ingestion pipeline (scrape → parse → chunk)
 - Sample chunks from 5 test documents
-

Week 3: Embedding and Vector Database

Objectives:

- Generate embeddings for document chunks
- Upsert embeddings to Qdrant

Tasks:

1. Create `lib/embedder.js`:
 - OpenAI embedding generation (text-embedding-3-small)
 - Batch processing (100 chunks per request)
2. Create `lib/qdrant-client.js`:
 - Qdrant upsert logic
 - Metadata schema setup
3. Extend `/api/ingest`:
 - Embed chunks
 - Upsert to Qdrant
4. Test ingestion with 10 sample documents
5. Verify Qdrant collection via Qdrant console

Deliverables:

- Embeddings generated for all sample chunks
 - Qdrant collection populated with 200+ chunks
-

Week 4: Retrieval and RAG Pipeline

Objectives:

- Implement semantic retrieval
- Build RAG generation logic

Tasks:

1. Create `lib/intent-classifier.js`:
 - Keyword-based classification
2. Create `lib/rag-pipeline.js`:
 - Query embedding
 - Qdrant retrieval (top-5 chunks)
 - Context aggregation
 - RAG prompt construction
 - LLM generation (OpenAI GPT-4)

3. Create `/api/chat` route:
 - Handle POST requests with user query
 - Orchestrate retrieval → generation
 - Return generated answer with citations
4. Test with 20 sample queries
5. Unit testing for retrieval and generation

Deliverables:

- Functional `/api/chat` endpoint
 - Citation-grounded answers for test queries
-

Week 5: Frontend Chat Interface

Objectives:

- Build chat UI
- Implement streaming responses

Tasks:

1. Create `app/components/ChatInterface.jsx`:
 - Chat message list
 - User input field and submit button
2. Create `app/components/MessageBubble.jsx`:
 - User and system message rendering
3. Create `app/components/StreamingText.jsx`:
 - Progressive token rendering
4. Implement Server-Sent Events (SSE) for streaming:
 - Modify `/api/chat` to stream tokens
 - Frontend consumes SSE stream
5. Style with Tailwind CSS (responsive design)
6. Test on desktop and mobile

Deliverables:

- Functional chat interface with streaming responses
 - Responsive design (desktop + mobile)
-

Week 6: Citation Display and Source Preview

Objectives:

- Implement citation rendering
- Build source preview panel

Tasks:

1. Create `app/components/CitationTag.jsx`:
 - Inline superscript citation numbers
 - Click handler to highlight source
2. Create `app/components/SourcePreview.jsx`:
 - Side panel (desktop) / modal (mobile)
 - Display document name, page, section, snippet
3. Create `lib/citation-extractor.js`:
 - Parse citations from LLM output
 - Validate against retrieved chunks
4. Extend `/api/chat` to return citation objects
5. Frontend integrates citations with source preview
6. Test citation click interactions

Deliverables:

- Inline citations with source preview
 - Functional citation validation
-

Week 7: Error Handling, Logging, and Testing

Objectives:

- Implement error handling
- Add logging and monitoring

- Conduct integration testing

Tasks:

1. Error handling:
 - API failures (OpenAI, Qdrant)
 - Network errors
 - Fallback responses
2. Logging:
 - Query logs (anonymized)
 - Retrieval logs (chunks, scores)
 - Generation logs (tokens, latency)
 - Ingestion audit logs
3. Create logging utility (`lib/logger.js`)
4. Integration testing:
 - 50 end-to-end test queries
 - Verify citation accuracy
 - Measure latency
5. Fix bugs and refine UX

Deliverables:

- Robust error handling across all endpoints
 - Comprehensive logging for monitoring
 - Test report with metrics
-

Week 8: Deployment, Documentation, and Evaluation

Objectives:

- Deploy to production (Vercel or equivalent)
- Complete documentation
- Conduct user evaluation

Tasks:

1. Deploy to Vercel:

- Configure environment variables
 - Set up CRON job for ingestion
 - Verify production deployment
2. Write documentation:
- README with setup instructions
 - API documentation
 - User guide
3. User evaluation:
- Recruit 10 test users
 - Collect feedback and ratings
 - Compute success metrics
4. Create final PRD presentation (slides)
5. Submit project deliverables

Deliverables:

- Live production deployment
- Complete documentation (README, API docs, user guide)
- User evaluation report
- Final presentation slides



Gantt Chart Summary

Week	Phase	Key Deliverables
1	Project Setup	Next.js app, cloud services configured
2	Document Ingestion	Scraping, parsing, chunking pipeline
3	Embedding & Vector DB	Embeddings generated, Qdrant populated
4	Retrieval & RAG	<code>/api/chat</code> endpoint, citation-grounded answers
5	Frontend Chat UI	Chat interface with streaming
6	Citations & Source Preview	Inline citations, source preview panel

Week	Phase	Key Deliverables
7	Error Handling & Testing	Robust error handling, integration tests
8	Deployment & Evaluation	Production deployment, user evaluation

15. Open Questions and Future Enhancements

15.1 Open Questions

15.1.1 Document Coverage

- **Question:** Which specific ICICI Bank documents should be prioritized for the MVP?
- **Recommendation:** Start with top 10 most frequently referenced documents (re-KYC guidelines, grievance policy, chargeback policy, home loan checklist, credit card terms).

15.1.2 Intent Classification Accuracy

- **Question:** Is keyword-based classification sufficient, or should we use LLM-based classification from the start?
- **Recommendation:** Start with keyword-based (simpler, faster); upgrade to LLM-based if accuracy <80%.

15.1.3 Chunking Strategy

- **Question:** Should we use fixed-size chunking or semantic chunking (e.g., paragraph-based)?
- **Recommendation:** Start with fixed-size (800 chars, 150 overlap); test semantic chunking if retrieval relevance <80%.

15.1.4 Deployment Platform

- **Question:** Should we deploy on Vercel, AWS, or Google Cloud?
- **Recommendation:** Vercel (fastest setup, built-in CRON support for Next.js).

15.2 Future Enhancements

15.2.1 Multi-Turn Conversations

- **Description:** Support follow-up questions that reference previous context.
- **Implementation:** Store conversation history in session state; pass full context to LLM.
- **Priority:** High (improves UX significantly).

15.2.2 User Feedback Loop

- **Description:** Allow users to rate answers (thumbs up/down) and provide comments.
- **Implementation:** Add rating buttons to each response; store feedback in database.
- **Priority:** Medium (useful for continuous improvement).

15.2.3 Admin Dashboard

- **Description:** Web interface for managing document sources, viewing logs, and monitoring metrics.
- **Implementation:** Build separate admin panel with authentication.
- **Priority:** Low (manual management sufficient for MVP).

15.2.4 Multi-Language Support

- **Description:** Support Hindi and other regional languages.
- **Implementation:** Multilingual embeddings (e.g., LaBSE); translate queries and responses.
- **Priority:** Medium (expands user base significantly).

15.2.5 Advanced Retrieval (Hybrid Search)

- **Description:** Combine semantic search with keyword search (BM25).
- **Implementation:** Use hybrid search in Qdrant or implement separate BM25 index.
- **Priority:** Medium (improves retrieval for specific terms like "Form 60").

15.2.6 Document Upload by Users

- **Description:** Allow users to upload their own documents (e.g., offer letters) for reference extraction.
- **Implementation:** Parse uploaded PDFs; create temporary embeddings.
- **Priority:** Low (out of scope for public documentation assistant).

15.2.7 Integration with ICICI Live Systems

- **Description:** Allow users to directly file grievances or submit forms via the chatbot.
- **Implementation:** Integrate with ICICI Bank APIs (requires official partnership).
- **Priority:** Low (requires legal and technical collaboration with ICICI Bank).

15.2.8 Voice Interface

- **Description:** Support voice queries and responses.
- **Implementation:** Speech-to-text (Whisper API); text-to-speech (TTS API).

- **Priority:** Low (nice-to-have for accessibility).

15.2.9 Personalized Recommendations

- **Description:** Suggest relevant documents based on user's query history.
- **Implementation:** Track user sessions; recommend documents using collaborative filtering.
- **Priority:** Low (conflicts with privacy-first approach).

15.2.10 Continuous Learning

- **Description:** Improve retrieval and generation based on user feedback.
- **Implementation:** Fine-tune embeddings or LLM on feedback data.
- **Priority:** Medium (long-term improvement strategy).

Appendices

Appendix A: Sample Document Sources

Document Type	Example URL	Intent Category
Re-KYC Guidelines	https://www.icicibank.com/customer-service/re-kyc	re-KYC
Video-KYC FAQ	https://www.icicibank.com/customer-service/video-kyc	re-KYC
Grievance Redressal Policy	https://www.icicibank.com/customer-service/grievance-redressal	grievance
Chargeback Policy	https://www.icicibank.com/customer-service/chargeback-policy	chargeback
Home Loan Checklist	https://www.icicibank.com/personal-banking/loans/home-loan	loan
Credit Card Terms	https://www.icicibank.com/personal-banking/cards/credit-card	credit-card

Appendix B: Sample Test Queries

1. "How do I complete Video-KYC if I keep getting errors?"
2. "What documents are needed to apply for a home loan?"
3. "How do I file a chargeback for an unauthorized transaction?"
4. "What is the escalation process for unresolved grievances?"

- 5. "Can I do re-KYC offline?"
- 6. "What is the timeline for chargeback dispute resolution?"
- 7. "How do I contact the Principal Nodal Officer?"
- 8. "What is the difference between salaried and self-employed loan documentation?"
- 9. "How do I download Form 60 for opening a bank account?"
- 10. "What should I do if my credit card dispute is not resolved in 30 days?"

Appendix C: Technology Stack Details

Component	Technology	Version/Details
Frontend Framework	Next.js	14.x (App Router)
Language	JavaScript	ES6+
Styling	Tailwind CSS	3.x
Backend	Next.js API Routes	Node.js 18+
Vector Database	Qdrant Cloud	Managed service
Embeddings	OpenAI text-embedding-3-small	1536 dimensions
LLM	OpenAI GPT-4 Turbo	Streaming enabled
Object Storage	AWS S3 or Google Cloud Storage	Standard tier
Scheduling	Vercel Cron or Node-Cron	Weekly runs
PDF Parsing	pdf-parse	Latest
HTML Parsing	cheerio	Latest
HTTP Client	axios or node-fetch	Latest

Appendix D: Glossary

Term	Definition
RAG	Retrieval-Augmented Generation: A technique that combines document retrieval with LLM generation

Term	Definition
	to produce citation-grounded answers.
Embedding	A vector representation of text in high-dimensional space, used for semantic similarity search.
Chunk	A segment of a document (typically 500-1000 characters) used as the unit of retrieval.
Intent	The category of a user query (e.g., re-KYC, grievance, loan).
Citation	A reference to a source document (name, page, section) that supports a generated answer.
Vector Database	A database optimized for storing and querying high-dimensional vectors (e.g., Qdrant, Pinecone).
Semantic Search	A search technique that retrieves documents based on meaning rather than exact keyword matches.
Streaming	Progressive delivery of LLM-generated tokens to the frontend as they are produced.
Fallback	A default response when the system cannot find relevant information in the corpus.
Ingestion	The process of downloading, parsing, chunking, embedding, and indexing documents.

Appendix E: References

1. **RAG Fundamentals:** Lewis et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." <https://arxiv.org/abs/2005.11401>
2. **Qdrant Documentation:** <https://qdrant.tech/documentation/>
3. **OpenAI Embeddings Guide:** <https://platform.openai.com/docs/guides/embeddings>
4. **OpenAI Streaming API:** <https://platform.openai.com/docs/api-reference/streaming>
5. **Next.js Documentation:** <https://nextjs.org/docs>
6. **Tailwind CSS Documentation:** <https://tailwindcss.com/docs>

Document Approval

Role	Name	Signature	Date
Product Manager	[Name]		

Role	Name	Signature	Date
Technical Architect	[Name]		
Engineering Lead	[Name]		
Legal/Compliance	[Name]		

End of Document